RELEASE IMPACT · FOR RELEASE & DELIVERY LEADS

Will this release change what production customers experience today?

Every release adds new logic for the new version. The one thing that can hurt live customers is if it *also* changes the path they're still running. Release Impact finds exactly that — automatically — and tells you what to test, who to ask, and whether it's been signed off.

Who runs what

A release doesn't move everyone at once. Two groups run in parallel:

NEW VERSION

Customers on the new release

Run the release's **new** logic. New fields and new steps live here.

LIVE / BAU

Customers still on the current version

Keep running the **previous** logic — **unchanged**. This is production, today.

New logic for new customers is expected and low-risk. The question that matters for go-live is about the **live** group.

The only risk that matters

Did this release change the path the **live** customers still run?

No — new version only

The change sits on the new version's path. Live customers are untouched.

Safe · no production impact

Yes — it reached the live path

The release changed the route or the message the live app still sends. Production behaviour changes.

Regression-test the current app before go-live · needs sign-off

That second case is a **backward-compatibility risk**: the shared back-end and existing customers must still work with the changed message. It's the highest-attention item in a release.

How the tool decides — no guesswork

1. **It knows which path each group runs.**
   The same version-routing the live system uses — so it looks at the exact route production still calls.
2. **It compares that path to its own previous version.**
   Using only *this release's* changes, and ignoring cosmetic edits (spacing, formatting, line endings) — the same "ignore whitespace" view a developer sees in their diff tool.
3. **No real change → not flagged. A real change → flagged.**
   A flagged item is marked **High risk + backward-compat**, with the exact lines that changed and the **person who changed them**.

Because it compares against the **previous version of that same file**, the verdict matches what a reviewer would confirm by hand — and a change made in an *earlier* release is never blamed on this one.

What you get, at a glance

A go/no-go summary — e.g. **3 APIs change a live (BAU) route** lead the list, ahead of routine new-version work. Each one tells you:

- **What changed** — Payload change or Route change (the actual before/after).
- **Who to ask** — the author who made the change.
- **How serious** — a **High** / BC marker (needs a backward-compatibility test).
- **Is it tested** — pass / fail / not-yet from the release test results.

Two views: a one-line-per-API **Summary** for leadership, and a **Detailed** view with the full changes — both export to **PDF** for the release sign-off pack.

Bottom line for a release decision: if nothing is flagged under "BAU route modified", the release is **clear of production impact** — it only affects the new version. If items are flagged, each is a **backward-compatibility check** the current app must pass before go-live, already attributed to an owner and a test result.

"Live / BAU" = customers still on the current version, whose transactions run the previous route unchanged · "BAU route modified" = this release changed that route or the message it sends, verified against the route's own previous version, cosmetic edits ignored.